

Contents

1	Azure OpenAI vs Anthropic - Architecture Comparison	1
1.1	Architecture Comparison Table	1
1.2	Detailed Component Comparison	2
1.2.1	1. Authentication	2
1.2.2	2. Stream Adapter	3
1.2.3	3. Detection & Routing	4
1.2.4	4. Payload Logging	5
1.3	File Structure Comparison	6
1.3.1	Azure OpenAI Files	6
1.3.2	Anthropic Files	6
1.4	Configuration Comparison	6
1.4.1	Azure OpenAI Config	6
1.4.2	Anthropic Config	7
1.5	Pros & Cons Summary	7
1.5.1	Azure OpenAI (Managed Identity)	7
1.5.2	Anthropic (API Key/Token)	7
1.6	When to Use Which?	8
1.6.1	Choose Azure OpenAI if:	8
1.6.2	Choose Anthropic if:	8
1.7	Migration Path	8
1.7.1	From Anthropic to Azure OpenAI:	8
1.7.2	From Azure OpenAI to Anthropic:	8
1.8	Hybrid Approach	8
1.9	Conclusion	9

1 Azure OpenAI vs Anthropic - Architecture Comparison



Figure 1: Diagram 1

1.1 Architecture Comparison Table

Component	Azure OpenAI	Anthropic
Auth Method	Managed Identity / Azure CLI	API Key / Setup Token / OAuth
Credential Provider	@azure/identity	Auth Profiles (direct key/token)
Token Management	Bearer token provider (auto-refresh)	Static API key/token
Client SDK	@azure/openai (native)	@anthropic-ai/sdk (via Pi AI)

Component	Azure OpenAI	Anthropic
Stream Adapter	Custom (streamAzureOpenAINative)	Default (streamSimple)
Detection Logic	provider === 'azureopenai' && auth === 'managedidentity'	provider === 'anthropic' (any auth)
Integration Type	Custom native integration	Standard Pi AI integration
Payload Logging	No	Yes (optional)
Dependencies	+2 (azure/identity, azure/openai)	0 (uses existing Pi AI)
Code Complexity	High (custom adapter)	Low (standard integration)

1.2 Detailed Component Comparison

1.2.1 1. Authentication

1.2.1.1 Azure OpenAI (Managed Identity)

```
// Step 1: Select credential
const credential = NODE_ENV === 'production'
  ? new ManagedIdentityCredential(clientId)
  : new AzureCliCredential()

// Step 2: Create bearer token provider
const tokenProvider = getBearerTokenProvider(
  credential,
  'https://cognitiveservices.azure.com/.default'
)

// Step 3: Initialize client
const client = new AzureOpenAI({
  endpoint: AZURE_OPENAI_ENDPOINT,
  azureADTokenProvider: tokenProvider,
  apiVersion: AZURE_OPENAI_API_VERSION,
})
```

Advantages: - Keyless authentication (no secrets in code) - Automatic token rotation - Azure AD integration - Enterprise-grade security

Disadvantages: - Complex setup (Azure IAM, RBAC) - Azure-specific (vendor lock-in) - Requires Azure infrastructure

1.2.1.2 Anthropic (API Key/Token)

```
// Step 1: Load credential
const credential = loadAuthProfile('anthropic:token:default')

// Step 2: Use directly
```

```
const apiKey = credential.token || process.env.ANTHROPIC_API_KEY
```

```
// Step 3: Call API (via Pi AI streamSimple)
```

```
// SDK handles auth automatically
```

Advantages: - Simple setup (just API key/token) - Cloud-agnostic - Easy to test and debug
- Multiple auth modes (API key, token, OAuth)

Disadvantages: - API key management (manual rotation) - Secrets in environment/config - No automatic refresh

1.2.2 2. Stream Adapter

1.2.2.1 Azure OpenAI (Custom Native Adapter)

```
// Custom adapter to fix tool arguments bug
```

```
export async function streamAzureOpenAINative(  
  context: AgentMessage[],  
  options?: StreamOptions,  
) : Promise<AssistantMessageEventStream> {  
  const eventStream = new AssistantMessageEventStream()  
  
  try {  
    // 1. Get native client  
    const client = await getClientInstance()  
  
    // 2. Convert Pi format → Native OpenAI format  
    const messages = convertContextToMessages(context)  
  
    // 3. Create streaming request  
    const stream = await client.chat.completions.create({  
      model: AZURE_OPENAI_DEPLOYMENT,  
      messages,  
      stream: true,  
    })  
  
    // 4. Handle stream events  
    for await (const chunk of stream) {  
      emitEvents(eventStream, chunk)  
    }  
  
    return eventStream  
  } catch (error) {  
    handleErrors(error)  
  }  
}
```

Purpose: Fix empty tool arguments bug in LangChain **Complexity:** High (custom event han-

dling) **Maintenance:** Requires updates when Pi AI changes

1.2.2.2 Anthropic (Default Adapter)

```
// Use default streamSimple from @mariozechner/pi-ai
import { streamSimple } from '@mariozechner/pi-ai'

// No custom adapter needed!
activeSession.agent.streamFn = streamSimple

// Optional: Wrap for logging
if (loggingEnabled) {
  activeSession.agent.streamFn = logger.wrapStreamFn(streamSimple)
}
```

Purpose: Standard Pi AI integration (no custom code) **Complexity:** Low (uses library) **Maintenance:** Automatic (library handles updates)

1.2.3 3. Detection & Routing

1.2.3.1 Azure OpenAI

```
const normalizedProvider = normalizeProviderId(provider)
const providerConfig = config.models.providers[normalizedProvider]

const usesAzureManagedIdentity =
  normalizedProvider === 'azureopenai' &&
  providerConfig.auth === 'managedidentity'

if (usesAzureManagedIdentity) {
  // Use custom native adapter
  activeSession.agent.streamFn = streamAzureOpenAINative
} else {
  // Use default
  activeSession.agent.streamFn = streamSimple
}
```

Trigger: Specific provider + specific auth mode **Adapter:** Custom native adapter **Complexity:** Conditional routing logic

1.2.3.2 Anthropic

```
const normalizedProvider = normalizeProviderId(provider)

// Always use default (no special case)
activeSession.agent.streamFn = streamSimple
```

```
// Optional: Add logging wrapper
if (shouldLogAnthropicPayload(provider)) {
  const logger = createAnthropicPayloadLogger(params)
  activeSession.agent.streamFn = logger.wrapStreamFn(streamSimple)
}
```

Trigger: Any Anthropic provider (any auth mode) **Adapter:** Default Pi AI adapter **Complexity:** Simple (no conditional routing)

1.2.4 4. Payload Logging

1.2.4.1 Azure OpenAI Not available

1.2.4.2 Anthropic Optional via environment variable

```
// Enable logging
export CLAWDBOT_ANTHROPIC_PAYLOAD_LOG=true
export CLAWDBOT_ANTHROPIC_PAYLOAD_LOG_FILE=/logs/anthropic-payload.jsonl

// Logger wraps streamFn
const logger = createAnthropicPayloadLogger({
  runId, sessionId, provider, modelId, workspaceDir
})

const wrappedStreamFn = logger.wrapStreamFn(streamSimple)

// Logs request payloads and usage
{
  "ts": "2026-02-09T20:00:00.000Z",
  "stage": "request",
  "payload": { ... },
  "payloadDigest": "sha256_hash"
}

{
  "ts": "2026-02-09T20:00:05.000Z",
  "stage": "usage",
  "usage": { input_tokens: 456, output_tokens: 123 }
}
```

Benefits: - Debug API issues - Track token usage - Audit requests - Monitor errors

1.3 File Structure Comparison

1.3.1 Azure OpenAI Files

```
src/agents/  
  azure-openai-models.ts          # Constants  
  azure-openai-native-client.ts   # Native client manager  
  azure-openai-stream-adapter-native.ts # Custom stream adapter  
  pi-embedded-runner/  
    run/  
      attempt.ts                  # Detection & routing
```

Total Custom Code: ~500 lines

1.3.2 Anthropic Files

```
src/agents/  
  anthropic-payload-log.ts        # Optional payload logger  
  pi-embedded-runner/  
    run/  
      attempt.ts                  # Standard routing
```

```
src/commands/  
  auth-choice.apply.anthropic.ts  # Auth management
```

Total Custom Code: ~300 lines (mostly logging, which is optional)

1.4 Configuration Comparison

1.4.1 Azure OpenAI Config

```
{  
  "models": {  
    "providers": {  
      "azureopenai": {  
        "auth": "managedidentity",  
        "baseUrl": "https://your-instance.openai.azure.com",  
        "api": "2024-12-01-preview",  
        "deployment": "gpt-4",  
        "managedIdentityClientId": "..."  
      }  
    }  
  }  
}
```

Environment Variables:

```
export AZURE_OPENAI_ENDPOINT=https://...  
export AZURE_OPENAI_DEPLOYMENT=gpt-4
```

```
export AZURE_OPENAI_API_VERSION=2024-12-01-preview
export AZURE_OPENAI_SCOPE=https://cognitiveservices.azure.com/.default
export MANAGED_IDENTITY_CLIENT_ID=...
```

1.4.2 Anthropic Config

```
{
  "models": {
    "providers": {
      "anthropic": {
        "auth": "token",
        "baseUrl": "https://api.anthropic.com",
        "api": "anthropic-messages"
      }
    }
  }
}
```

Environment Variables:

```
export ANTHROPIC_API_KEY=sk-ant-api03-...

# Optional logging
export CLAWDBOT_ANTHROPIC_PAYLOAD_LOG=true
export CLAWDBOT_ANTHROPIC_PAYLOAD_LOG_FILE=/logs/anthropic.jsonl
```

1.5 Pros & Cons Summary

1.5.1 Azure OpenAI (Managed Identity)

Pros: - Keyless authentication (most secure) - Automatic token rotation - Enterprise-grade security - Azure AD integration - RBAC support - Audit logs via Azure

Cons: - Complex setup (IAM, RBAC) - Azure vendor lock-in - Requires Azure infrastructure - Custom adapter maintenance - More dependencies - Higher code complexity

Best For: - Enterprise Azure deployments - Zero-trust security requirements - Regulated industries (finance, healthcare) - Large teams with dedicated Azure admins

1.5.2 Anthropic (API Key/Token)

Pros: - Simple setup (just API key) - Cloud-agnostic - Easy to test and debug - Standard Pi AI integration - Fewer dependencies - Optional payload logging - Multiple auth modes

Cons: - API key management (manual rotation) - Secrets in environment/config - No automatic token refresh - Limited enterprise IAM integration

Best For: - General-purpose AI integration - Rapid development - Multi-cloud deployments - Small to medium teams - Projects without Azure infrastructure

1.6 When to Use Which?

1.6.1 Choose Azure OpenAI if:

1. You're already on Azure infrastructure
2. You need keyless authentication
3. You have enterprise security requirements
4. You need Azure AD integration
5. You have dedicated Azure admins

1.6.2 Choose Anthropic if:

1. You want simple, cloud-agnostic integration
 2. You're not on Azure infrastructure
 3. You want rapid development
 4. You need flexibility (multi-cloud, local dev)
 5. You want to minimize dependencies
-

1.7 Migration Path

1.7.1 From Anthropic to Azure OpenAI:

1. Set up Azure infrastructure (Resource Group, OpenAI resource)
2. Configure Managed Identity (assign RBAC roles)
3. Add Azure dependencies (@azure/identity, @azure/openai)
4. Implement custom stream adapter
5. Update detection logic in PiEmbeddedRunner
6. Test with Azure CLI credentials (dev)
7. Deploy with Managed Identity (prod)

1.7.2 From Azure OpenAI to Anthropic:

1. Generate Anthropic API key or setup-token
 2. Store in auth profile or environment variable
 3. Update provider config (change auth mode)
 4. Remove Azure-specific code (if desired)
 5. Remove Azure dependencies (optional)
 6. Test with standard Pi AI integration
-

1.8 Hybrid Approach

You can support **both** Azure OpenAI and Anthropic in the same application:


```

const normalizedProvider = normalizeProviderId(provider)
const providerConfig = config.models.providers[normalizedProvider]

if (normalizedProvider === 'azureopenai' && providerConfig.auth === 'managedidentity') {
  // Use custom Azure adapter
  activeSession.agent.streamFn = streamAzureOpenAINative
} else if (normalizedProvider === 'anthropic' && loggingEnabled) {
  // Use default adapter with logging
  const logger = createAnthropicPayloadLogger(params)
  activeSession.agent.streamFn = logger.wrapStreamFn(streamSimple)
} else {
  // Use default adapter
  activeSession.agent.streamFn = streamSimple
}

```

Benefits: - Flexibility: Use Azure in production, Anthropic in dev - Vendor diversity: Avoid single-vendor lock-in - Failover: Switch providers if one is down - Cost optimization: Use cheaper provider for non-critical tasks

1.9 Conclusion

Both architectures serve different needs:

- **Azure OpenAI:** Enterprise-grade, keyless, complex
- **Anthropic:** Simple, flexible, developer-friendly

Choose based on your **security requirements, infrastructure, and team expertise.**